

## MINOR ASSIGNMENT-07 — SOLUTION

Game Programming with C++ (CSE 3545) | CO6 | Bullet Class, Collision & Inheritance

---

### Q1. SFML class for bullet visual and private member declaration:

```
// SFML class: RectangleShape (used to visually represent a bullet)

class Bullet {
private:
    RectangleShape m_Shape;    // visual representation of the bullet
    Vector2f       m_Position;
    float          m_BulletSpeedX = 0;
    float          m_BulletSpeedY = 0;
    bool           m_InFlight = false;
    float          m_MaxX, m_MinX, m_MaxY, m_MinY;
};
```

### Q2. Public member function prototypes for Bullet class:

```
class Bullet {
public:
    Bullet(); // constructor

    // Set starting position and direction of travel
    void shoot(float startX, float startY,
               float xTarget, float yTarget);

    // Return the bounding box for collision detection
    FloatRect getPosition();

    // Return the sprite/shape for drawing
    RectangleShape getShape();

    // Update bullet position each frame
    void update(float elapsedTime);

    // Check if the bullet is still in flight
    bool isInFlight();

    // Stop the bullet
    void stop();
};
```

### Q3. Bullet constructor — size 20×20, color red:

```
Bullet::Bullet() {
    m_Shape.setSize(Vector2f(20.0f, 20.0f));
    m_Shape.setFillColor(Color::Red);
}
```

### Q4. Set bullet position and calculate gradient of travel:

```
// Set the starting position of the bullet
m_Position.x = startX;
m_Position.y = startY;
m_Shape.setPosition(m_Position);

// Calculate gradient of the flight path
float gradient = (targetY - startY) / (targetX - startX);
// gradient = rise / run = dy / dx
```

#### Q5. Compute horizontal and vertical speed from gradient and bullet speed:

```
// Using Pythagoras: total speed vector has magnitude m_BulletSpeed
// If gradient = dy/dx, then:
//   direction vector = (1, gradient), magnitude = sqrt(1 + gradient^2)
//   normalize and scale by m_BulletSpeed

float magnitude = sqrt(1 + gradient * gradient);

m_BulletSpeedX = m_BulletSpeed / magnitude;
m_BulletSpeedY = m_BulletSpeed * gradient / magnitude;

// If target is to the left, negate both:
if (targetX < startX) {
    m_BulletSpeedX = -m_BulletSpeedX;
    m_BulletSpeedY = -m_BulletSpeedY;
}
```

#### Q6. Set maximum range of 1200 pixels for bullet:

```
// Maximum range: 1200 pixels in any direction from start
m_MaxX = startX + 1200;
m_MinX = startX - 1200;
m_MaxY = startY + 1200;
m_MinY = startY - 1200;
```

#### Q7. Test if bullet has exceeded max range, set m\_InFlight=false:

```
void Bullet::update(float elapsedTime) {
    // Update position
    m_Position.x += m_BulletSpeedX * elapsedTime;
    m_Position.y += m_BulletSpeedY * elapsedTime;
    m_Shape.setPosition(m_Position);

    // Check if out of range
    if (m_Position.x > m_MaxX || m_Position.x < m_MinX ||
        m_Position.y > m_MaxY || m_Position.y < m_MinY) {
        m_InFlight = false;
    }
}
```

#### Q8. Handle left mouse button click to fire a bullet:

```
// Place inside the event poll loop (while window.pollEvent(event))
if (event.type == Event::MouseButtonPressed) {
    if (event.mouseButton.button == Mouse::Left) {
        // Fire a bullet from player position toward mouse position
        if (bulletsInClip > 0) {
            bullets[currentBullet].shoot(
                player.getCenter().x, player.getCenter().y,
                mouseWorldPosition.x, mouseWorldPosition.y);
            currentBullet++;
            if (currentBullet >= 100) currentBullet = 0;
            bulletsInClip--;
        }
    }
}
// This code belongs in the EVENT HANDLING section of the main game loop.
```

#### Q9. Loop through bullets array, check in-flight, draw if active:

```
for (int i = 0; i < 100; i++) {
    if (bullets[i].isInFlight()) {
        window.draw(bullets[i].getShape());
    }
}
```

#### Q10. Test if player touched by zombie; change state if health $\leq 0$ :

```
for (int i = 0; i < numZombies; i++) {
    if (player.getPosition().intersects(zombies[i].getPosition())) {
        // Reduce player health
        player.hit();
        // Play hit sound
        splat.play();

        if (player.getHealth() <= 0) {
            state = State::GAME_OVER;
            // Display game over screen / message
        }
    }
}
```

#### Q11. Test if zombie is shot; change state if all zombies dead:

```

for (int i = 0; i < 100; i++) {           // loop bullets
    for (int j = 0; j < numZombies; j++) { // loop zombies
        if (bullets[i].isInFlight() &&
            bullets[i].getPosition().intersects(zombies[j].getPosition())) {

            bullets[i].stop(); // bullet hits zombie

            if (zombies[j].hit()) { // zombie takes damage; returns true if dead
                score++;
                scoreText.setString("Score: " + to_string(score));
                numZombiesAlive--;

                if (numZombiesAlive == 0) {
                    state = State::LEVELING_UP; // all zombies dead
                }
            }
        }
    }
}

```

**Q12. Class B publicly derived from A (A has 4 public, B has 1 public):**

```

class A {
public:
    void func1();
    void func2();
    void func3();
    void func4();
};

class B : public A { // public derivation
public:
    void func5();    // B's own function
};

// Class B can use: func1, func2, func3, func4 (inherited from A as public)
//                  + func5 (its own)
// Total public functions accessible by B's object: 5

```

**Q13. Class B privately derived from A (A has 4 public, B has 1 private):**

```

class A {
public:
    void func1();
    void func2();
    void func3();
    void func4();
};

class B : private A { // private derivation
private:
    int m_Data;        // B's own private member
};

// When privately derived: A's public members become private in B.
// B itself can use func1..func4 internally, but outside code cannot.
// Total private members accessible within B: 4 (inherited) + 1 (own) = 5

```

**Q14. Class E inherited from A(public), B(private), C(protected), D(public):**

```
class A { /* ... */ };
class B { /* ... */ };
class C { /* ... */ };
class D { /* ... */ };

class E : public    A,
         private   B,
         protected C,
         public    D {
    // E's own members
};
```

**Q15. Output of Derived d with multiple inheritance (B1, B2, D):**

**Note:** The code has a typo — the derived class name is missing. Assuming: class D : public B1, public B2 { ... }; and Derived d; means D d;

```
B1
B 2
D
```

**Explanation:** In multiple inheritance, base class constructors are called in the order they are listed. B1 constructor called first → 'B1'; B2 constructor → 'B 2'; D constructor → 'D'. Output: **B1 / B 2 / D** (one per line)